MySQL Data Types

Complete Practice Assignment

Beginner	Intermediate	Advanced	Tricky
25 Questions · 50 Marks	12 Questions · 40 Marks	10 Questions · 40 Marks	8 Questions · 30 Marks

Total: 55 Questions | 160 Marks | Topics: All MySQL Data Types

Instructions for Students:

1. Write all CREATE TABLE statements with the exact data types specified.
2. Insert at least 3 rows per table unless a different number is stated.
3. Write and run all SELECT queries. Paste the actual output as a comment.
4. Beginner section is compulsory for all students.
5. Intermediate and Advanced sections are for higher grades.
6. Tricky questions need written explanations — not just code.
7. Use utf8mb4 charset unless told otherwise.

SECTION 1 — BEGINNER LEVEL [50 Marks]

Topics covered: INT, TINYINT, SMALLINT, BIGINT, VARCHAR, CHAR, DATE, BOOLEAN, DECIMAL, FLOAT, TIME, YEAR, basic SELECT / INSERT / WHERE / ORDER BY

Part A — Understanding Data Types (Theory + Simple Queries)

Answer the following questions. Some need only a SQL statement, some need a short written answer.

Q#	Question	Marks	Hint
1	What is the difference between CHAR(10) and VARCHAR(10)? Give one real-life example for each.	2	Fixed vs variable length
2	Which data type would you use to store: (a) a student age, (b) a product price, (c) a blog post body, (d) a yes/no flag? Write the type name only.	2	TINYINT, DECIMAL, TEXT, BOOLEAN
3	Write a SELECT query to check the current date in MySQL.	1	SELECT CURDATE()
4	Write a SELECT query that calculates 7 * 8 without using any table.	1	SELECT 7 * 8
5	What is the maximum value that can be stored in a TINYINT UNSIGNED column?	1	0 to 255

Part B — Students Table (INT, VARCHAR, DATE, BOOLEAN, TINYINT)

Create the following table exactly as specified. Then complete every task below.

Column	Data Type	Constraint	Description
student_id	INT	AUTO_INCREMENT PRIMARY KEY	Unique ID for each student
first_name	VARCHAR(60)	NOT NULL	First name
last_name	VARCHAR(60)	NOT NULL	Last name
age	TINYINT UNSIGNED	NOT NULL	Age, range 0-255
city	VARCHAR(80)	DEFAULT "Mumbai"	Home city
enroll_date	DATE	NOT NULL	Date of enrollment
is_active	BOOLEAN	DEFAULT TRUE	Active student flag

Q#	Question	Marks	Hint
1	Write the complete CREATE TABLE statement for the students table	2	Check all columns

	above.		and types
2	Insert 5 student rows. Rules: at least 2 from Mumbai, at least 1 is_active = FALSE, ages must vary (18-30 range).	3	5 INSERT rows total
3	SELECT all columns for every student.	1	SELECT * FROM ...
4	SELECT only first_name, last_name and city for all students.	1	SELECT specific columns
5	SELECT all students from Mumbai.	1	WHERE city = 'Mumbai'
6	SELECT all active students (is_active = TRUE) sorted by last_name A to Z.	2	WHERE + ORDER BY ASC
7	SELECT all students whose age is between 20 and 25 (inclusive).	1	BETWEEN ... AND ...
8	SELECT the student with the earliest enroll_date.	2	ORDER BY enroll_date ASC LIMIT 1
9	Count how many students are currently active.	1	SELECT COUNT(*) WHERE is_active = TRUE
10	UPDATE the city of student_id = 1 to "Delhi". Then SELECT that student to confirm the change.	2	UPDATE ... SET ... WHERE

Part C — Products Table (VARCHAR, DECIMAL, SMALLINT, INT)

A small online shop needs a products table. Build it from scratch.

Q#	Question	Marks	Hint
1	Create a table products with: product_id (INT, PK, AUTO_INCREMENT), name (VARCHAR 150, NOT NULL), description (VARCHAR 300), price (DECIMAL 10,2, NOT NULL), stock_qty (SMALLINT UNSIGNED DEFAULT 0), category (VARCHAR 60).	2	DECIMAL(10,2) for money always
2	Insert 5 products across 3 different categories. Include: one product with price = 0.00, one with stock_qty = 0.	3	Edge case rows are important
3	SELECT all products where price is between 100 and 5000.	1	BETWEEN
4	SELECT the 3 most expensive products (name and price only).	2	ORDER BY price DESC LIMIT 3
5	SELECT count of products grouped by category. Show category name and count.	2	GROUP BY category
6	Calculate the average price of all products using AVG().	1	SELECT AVG(price)

Part D — Library Books Table (YEAR, DATE, CHAR, SMALLINT)

A library needs to track its book collection.

Q#	Question	Marks	Hint
1	Create table books: book_id (INT PK AUTO_INCREMENT), title (VARCHAR 200 NOT NULL), author (VARCHAR 100 NOT NULL), isbn (CHAR 13 NOT NULL UNIQUE — always 13 characters), pub_year (YEAR), total_copies (TINYINT UNSIGNED DEFAULT 1), available_copies (TINYINT UNSIGNED DEFAULT 1).	3	CHAR for fixed-length ISBN. YEAR for publication year
2	Insert 4 books. Include one book published before 2000 and one with available_copies = 0 (all checked out).	2	YEAR accepts 1901-2155
3	SELECT all books published after 2010, sorted by pub_year newest first.	1	WHERE pub_year > 2010
4	SELECT books that are fully checked out (available_copies = 0).	1	WHERE available_copies = 0
5	SELECT title and (total_copies - available_copies) AS checked_out for all books.	2	Arithmetic in SELECT

Part E — Daily Attendance Table (TIME, DATE, BOOLEAN)

Track employee attendance with exact clock-in and clock-out times.

Q#	Question	Marks	Hint
1	Create table attendance: att_id (INT PK AUTO_INCREMENT), emp_name (VARCHAR 100 NOT NULL), att_date (DATE NOT NULL), clock_in (TIME), clock_out (TIME), is_present (BOOLEAN DEFAULT TRUE).	2	TIME stores HH:MM:SS
2	Insert records for 3 employees over 2 days (6 rows total). One employee was absent — set is_present = FALSE and both times to NULL.	3	NULL is allowed for absent rows
3	SELECT all attendance records for today using CURDATE().	1	WHERE att_date = CURDATE()
4	SELECT emp_name and TIMEDIFF(clock_out, clock_in) AS hours_worked for all present employees.	2	TIMEDIFF() calculates time difference
5	SELECT all employees who clocked in before 09:00:00.	1	WHERE clock_in <'09:00:00'

SECTION 2 — INTERMEDIATE LEVEL [40 Marks]

Topics: ENUM, SET, TEXT, MEDIUMTEXT, DATETIME, TIMESTAMP, BLOB, BINARY, JSON

Q1 — Order Management (ENUM, DECIMAL, DATETIME, TIMESTAMP)

Build an order table that demonstrates the difference between DATETIME and TIMESTAMP.

Q#	Question	Marks	Hint
1	Create table orders: order_id (INT PK AUTO_INCREMENT), customer (VARCHAR 100), status (ENUM 'pending','confirmed','shipped','delivered','cancelled' DEFAULT 'pending'), total (DECIMAL 12,2), scheduled_at (DATETIME — the delivery slot), created_at (TIMESTAMP DEFAULT CURRENT_TIMESTAMP), updated_at (TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP).	4	Two time columns for comparison
2	Insert 4 orders with different statuses. Include one cancelled.	2	Use all 4 status values
3	UPDATE one order to status 'shipped'. Then SELECT updated_at. Has it changed? Explain why in one sentence.	3	ON UPDATE CURRENT_TIMESTAMP auto-fires
4	Change session timezone to '+05:30'. SELECT scheduled_at and created_at. Then change to UTC. Which column changes its displayed value? Why?	3	DATETIME never converts; TIMESTAMP always does

Q2 — Blog Articles (TEXT, MEDIUMTEXT, ENUM, SET, FIND_IN_SET)

Q#	Question	Marks	Hint
1	Create table articles: article_id (INT PK), title (VARCHAR 200 NOT NULL), summary (TINYTEXT), body (MEDIUMTEXT), status (ENUM 'draft','published','archived' DEFAULT 'draft'), tags (SET 'mysql','python','docker','linux','sql'), author (VARCHAR 100).	3	TINYTEXT ≤ 255 bytes
2	Insert 3 articles: one draft with no tags, one published with tags 'mysql,sql', one archived.	2	SET stores comma-separated values
3	Find all articles that contain 'mysql' in tags using FIND_IN_SET. Explain why WHERE tags = 'mysql' would NOT work.	3	FIND_IN_SET vs = operator
4	SELECT title and CHAR_LENGTH(body) AS body_chars for all articles, ordered by body_chars DESC.	2	CHAR_LENGTH counts characters

Q3 — Document Storage (BINARY, VARBINARY, BLOB)

Q#	Question	Marks	Hint
1	Create table docs: doc_id (INT PK AUTO_INCREMENT), filename (VARCHAR 255), file_type (ENUM 'pdf','jpg','docx'), size_bytes (INT UNSIGNED), checksum (BINARY(32)), uploaded_at (TIMESTAMP DEFAULT CURRENT_TIMESTAMP).	3	BINARY(32) for SHA-256 hash
2	Insert 2 rows using UNHEX(SHA2(filename, 256)) for checksum. SELECT HEX(checksum) to verify.	3	HEX() to read binary
3	Find all PDF files larger than 1 MB (1048576 bytes).	2	WHERE file_type AND size_bytes >

Q4 — User Preferences (JSON, JSON_SET, JSON_REMOVE)

Q#	Question	Marks	Hint
1	Create table user_prefs: user_id (INT PK), username (VARCHAR 60), prefs (JSON). Insert 3 users. Each prefs must have: theme, language, notifications (bool), font_size (number).	3	JSON: {"theme":"dark",...}
2	SELECT username and prefs->>'$.theme' AS theme for all users. Explain the difference between -> and ->>.	2	->> removes quotes, -> keeps JSON type
3	Use JSON_SET to change theme to 'light' for user_id 1 and add a new key 'sidebar': true.	2	JSON_SET(prefs, path, value)
4	Use JSON_REMOVE to delete the 'notifications' key from user_id 2's prefs.	2	JSON_REMOVE(col, path)

SECTION 3 — ADVANCED LEVEL [40 Marks]

Topics: Generated columns (VIRTUAL + STORED), JSON indexing, BIT flags, UUID pattern, combined design

Q1 — Product Catalog with Generated Columns

Q#	Question	Marks	Hint
1	Create table catalog: product_id (INT PK), name (VARCHAR 150), original_price (DECIMAL 10,2), discount_pct (DECIMAL 5,2 DEFAULT 0). Add VIRTUAL generated column final_price = ROUND(original_price*(1-discount_pct/100),2). Add VIRTUAL generated column you_save = ROUND(original_price*discount_pct/100,2).	5	Generated columns use AS(...) VIRTUAL
2	Insert 4 products. Then try to INSERT a value directly into final_price. What error appears? Why?	3	Cannot insert into generated column
3	UPDATE discount_pct of one product. Re-SELECT final_price and you_save — do they update automatically? Explain why.	4	VIRTUAL recalculates every SELECT

Q2 — JSON Indexing with STORED Generated Columns

Q#	Question	Marks	Hint
1	Create table products_v2: product_id (INT PK), name VARCHAR(150), metadata JSON. Add STORED generated column brand = metadata->>'$.brand'. Add STORED generated column price = CAST(metadata->>'$.price' AS DECIMAL(10,2)). Add INDEX on brand. Add INDEX on price.	5	STORED can be indexed; VIRTUAL cannot
2	Insert 5 rows. Each metadata must include brand, price, and a tags array e.g. ["gaming","laptop"].	2	Valid JSON required
3	Explain why WHERE brand='Dell' is faster than WHERE metadata->>'$.brand'='Dell' on 1 million rows.	3	Index on column vs full JSON scan
4	Find all products where the tags JSON array contains 'gaming' using JSON_CONTAINS.	3	JSON_CONTAINS(col,'"gaming"','$.tags')

Q3 — BIT Flags for Permission System

Q#	Question	Marks	Hint
1	Create table user_roles: user_id (INT PK), username VARCHAR(60), permissions BIT(8) DEFAULT b'00000000'. Bit positions: 1=read, 2=write, 3=delete, 4=export, 5=admin.	3	BIT(8) = 8 independent switches
2	Insert 4 users: superadmin (all 5 bits ON), editor (read+write), viewer (read only), manager (read+write+export).	3	b'00011111' for all 5 ON
3	SELECT username and (permissions & b'00000100')>0 AS can_delete for all users. List what each row returns.	3	Bitwise AND isolates one bit
4	SELECT permissions+0 AS numeric_value for all users. Explain what number each user gets and why.	3	Powers of 2 add up

SECTION 4 — TRICKY QUESTIONS [30 Marks]

These questions test common traps. Read very carefully. Written explanation is required for full marks.

Tricky Q1 — VARCHAR Size Trap

```
CREATE TABLE test_varchar (
  id    INT,
  name  VARCHAR(65535)
);
```

Q#	Question	Marks	Hint
1	Run the above with utf8mb4 charset. Does it succeed? Write the exact error code and message.	2	Error 1074
2	Explain in 2 sentences WHY it fails. What is the real limit for VARCHAR with utf8mb4?	3	Row = 65535 bytes. utf8mb4 = 4 bytes per char
3	Show two correct solutions: (a) correct VARCHAR(n), (b) use TEXT instead.	2	(65535-2)/4 = 16383

Tricky Q2 — FLOAT vs DECIMAL Money Error

```
CREATE TABLE price_test (f FLOAT, d DECIMAL(10,2));
INSERT INTO price_test VALUES (9.99, 9.99);
SELECT f*3 AS float_result, d*3 AS decimal_result FROM price_test;
```

Q#	Question	Marks	Hint
1	Run the query. What are the exact values of float_result and decimal_result?	2	One is not 29.97
2	Explain WHY FLOAT gives a different result. Use the words "binary fraction" in your answer.	3	0.1 cannot be exact in binary
3	A teacher calculates final marks: 1000 students each scored 9.99. Write both FLOAT and DECIMAL totals. Which one should be used for reports?	3	9.99 * 1000

Tricky Q3 — ENUM ORDER BY Trap

```
CREATE TABLE tasks (
```

```
  task   VARCHAR(60),
  level  ENUM('critical','high','medium','low')
);
INSERT INTO tasks VALUES
  ('Deploy app', 'high'),
  ('Fix crash',  'critical'),
  ('Write docs', 'low'),
  ('Code review','medium');
SELECT * FROM tasks ORDER BY level;
```

Q#	Question	Marks	Hint
1	What is the ORDER BY output order? List all 4 rows in the order MySQL returns them. Is it alphabetical?	2	critical=1 high=2 medium=3 low=4
2	Write a query to sort these tasks alphabetically by level name instead of by ENUM index.	2	CAST(level AS CHAR)
3	What happens when you INSERT level='urgent' (not in the ENUM list)? Test with strict mode ON and OFF. Show both outputs.	4	SET sql_mode='STRICT_TRANS_TABLES'

Tricky Q4 — CURDATE() in Generated Column

```
CREATE TABLE student_age (
  id         INT PRIMARY KEY,
  name       VARCHAR(100),
  birth_date DATE NOT NULL,
  age        TINYINT UNSIGNED GENERATED ALWAYS AS
             (TIMESTAMPDIFF(YEAR, birth_date, CURDATE())) VIRTUAL
);
```

Q#	Question	Marks	Hint
1	Run this CREATE TABLE. What is the exact error code and message?	2	Error 3763
2	In simple words, explain why CURDATE() is banned in a generated column. What does "non-deterministic" mean here?	3	Same row must always give same result
3	Write the correct solution — keep the birth_date column but calculate age in the SELECT query instead. Show a full working example with INSERT and SELECT.	3	TIMESTAMPDIFF(YEAR, birth_date, CURDATE()) in SELECT

Tricky Q5 — CHAR Padding Comparison

```
CREATE TABLE char_test (code CHAR(5));
INSERT INTO char_test VALUES ('AB');
```

```
SELECT * FROM char_test WHERE code = 'AB';
SELECT * FROM char_test WHERE code = 'AB   ';
SELECT LENGTH(code) AS len FROM char_test;
```

Q#	Question	Marks	Hint
1	Both WHERE clauses above return the same row. Why? What does MySQL do to trailing spaces in CHAR comparison?	3	CHAR pads with spaces but strips on compare
2	Does SELECT LENGTH(code) return 2 or 5? Explain the reason.	2	LENGTH returns stored characters not defined width
3	How is BINARY(5) different from CHAR(5) in terms of padding and comparison? Give a short code example.	3	BINARY pads 0x00 and does NOT trim on compare

Quick Reference — MySQL Data Types Cheat Sheet

Integer Types

Type	Bytes	Signed Range	Unsigned Range	Best used for
TINYINT	1	-128 to 127	0 to 255	Age, rating, flag
SMALLINT	2	-32,768 to 32,767	0 to 65,535	Seat count, small quantities
INT	4	-2.1 B to 2.1 B	0 to 4.2 B	IDs, quantities — DEFAULT choice
BIGINT	8	±9.2 x 10^18	0 to 18.4 x 10^18	File sizes, huge counters
DECIMAL(p,s)	varies	Exact fixed-point	Same	MONEY — always use for currency
FLOAT / DOUBLE	4 / 8	Approximate	Same	Sensors, GPS — NOT money

String Types

Type	Max Size	Key Fact
CHAR(n)	255 chars	Fixed length — always pads to n chars. Fast for codes.
VARCHAR(n)	16,383 (utf8mb4)	Variable length. Most common. Use for names, emails, titles.
TEXT	64 KB	Off-row storage. No DEFAULT. Use for comments and descriptions.
MEDIUMTEXT	16 MB	For full articles and HTML pages.
ENUM	65,535 options	Pick ONE value. Stored as index. Sorts by definition order NOT alphabetically.
SET	64 options	Pick MULTIPLE values. Stored as bitmask. Query with FIND_IN_SET().

Date and Time Types

Type	Bytes	Format	Key Fact
DATE	3	YYYY-MM-DD	Date only. No time. For birthdays and deadlines.
TIME	3	HH:MM:SS	Can exceed 24 hours. Range: -838:59:59 to 838:59:59.
DATETIME	5	YYYY-MM-DD	Stores as-is. No timezone conversion. Range up to

		HH:MM:SS	year 9999.
TIMESTAMP	4	YYYY-MM-DD HH:MM:SS	Stores UTC. Auto timezone convert on read. Limit: 2038.
YEAR	1	YYYY	Year only. Range 1901 to 2155. Smallest date type.

Marking Scheme

Section	Level	Marks	Key Topics
Section 1 Parts A-E	**Beginner**	50	INT, VARCHAR, DATE, DECIMAL, BOOLEAN, TIME, YEAR
Section 2 Q1-Q4	**Intermediate**	40	ENUM, SET, TEXT, TIMESTAMP, BLOB, JSON
Section 3 Q1-Q3	**Advanced**	40	Generated columns, JSON index, BIT flags
Section 4 Q1-Q5	**Tricky**	30	Edge cases, traps, written explanations
TOTAL	**All Levels**	**160**	**Full MySQL Data Types Coverage**

Grade Scale: 144-160 = Excellent | 112-143 = Good | 80-111 = Average | Below 80 = Needs Practice